CN451 Network Programming

Mini-Project 2: Socket Programming Experiment with Python

Dr. Mohamed MUSBAH

By doing the tasks of this project, you should learn how to use Python to

- Understand and apply **`socket`** library.
- Execute tasks on a remote device.
- Read and write data on external files on another device.
- Execute tasks from multiple clients.
- Send and Receive files.

Important Note: Do not forget to use (**help**) and (**dir**) commands for more help.

Scenario: You are asked to develop a multi-feature chat application that enables two (or more) machines on a network to communicate in a client-server model. Use **`socket`** library (jointly with other libraries you may need to use) to carry-out the communications. The requirements of this mini-app are:

Project Tasks

Task 1: Each client has to register with the server. And clients can send and receive messages to the server (with an echo so that every client can see the conversation from all clients)

Task 2: Add a game feature for two clients; a rock-paper-scissors game so that the players play five times, and the winner who wins three of them (the game ends when a winner exists)

Task 3: Add another feature to this application so that each client can upload a small file (of no more than 150 KB in size-for example; pdf or image) to the server. Also, each client can download a file from the listed files on the server (the files uploaded by all clients).

Task 4: Add another feature (of your choice) to this application that uses the communications between client and server

**** You have complete flexibility in designing the commands, responses, warnings, and error messages for each of the aforementioned tasks, tailoring them as you see fit.*

Submission Requirements

You **MUST** submit the following three items via Google Classroom:

1- Python Scripts: Your source code as a **`*.py`** files.

2- Application Report (User Guide-Like): A Google Docs file that documents the whole app and how every component works (No code, just use case and how to use).

3- Verification Video (Google Vids **LINK**): A short (3–4 minutes) screen recording with your voice (and face). The video MUST have a sensible name. In the video, you must:

- Briefly show your code.
- Run the script in real-time.
- Explain your new feature. What it does and how it works?

Submissions without a video will NOT be graded to ensure academic integrity.

Late submissions will be penalized ONE mark per day.